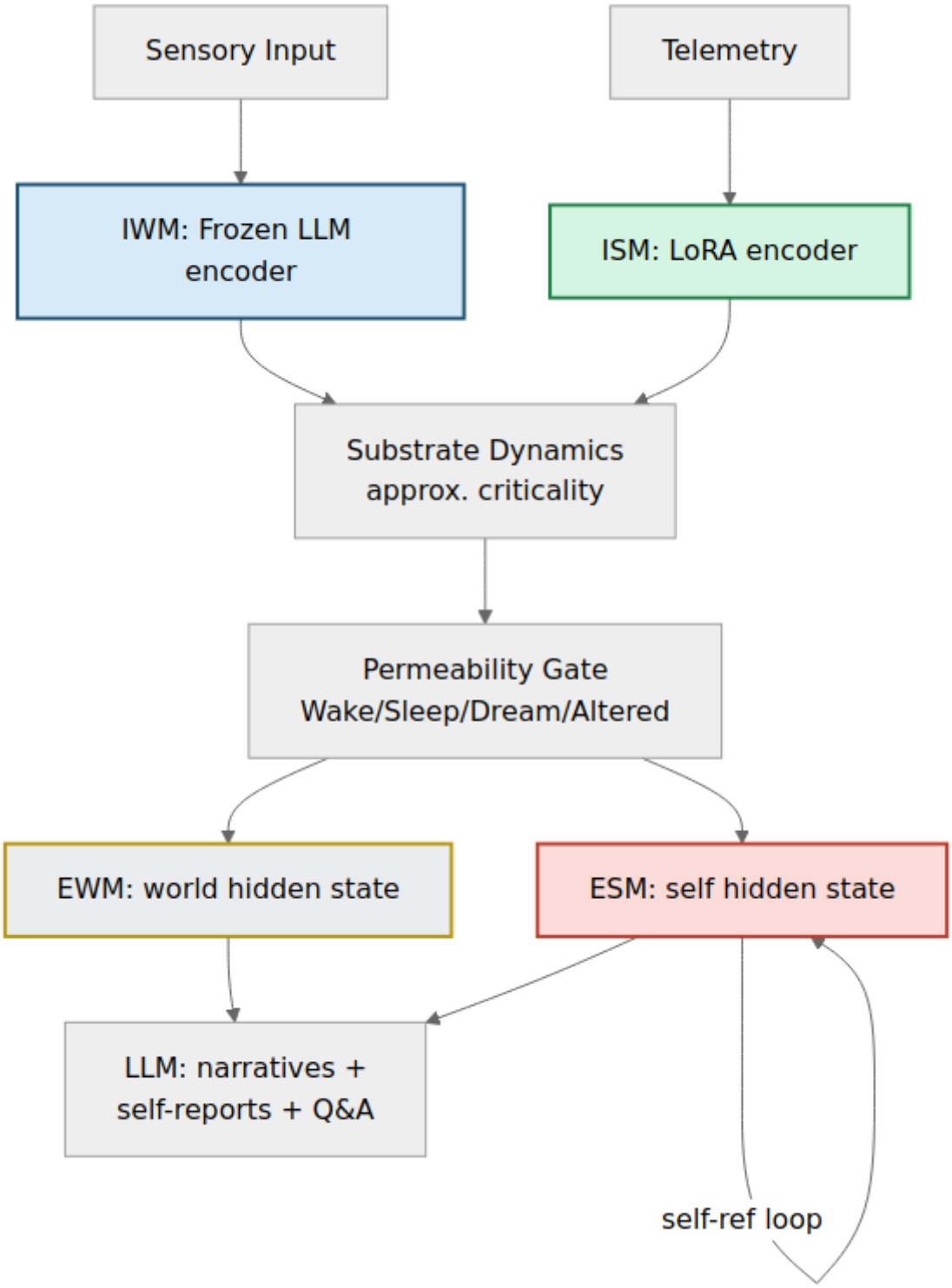
Design 16: The Mirror Box (Quick-Win Pseudo-AC)

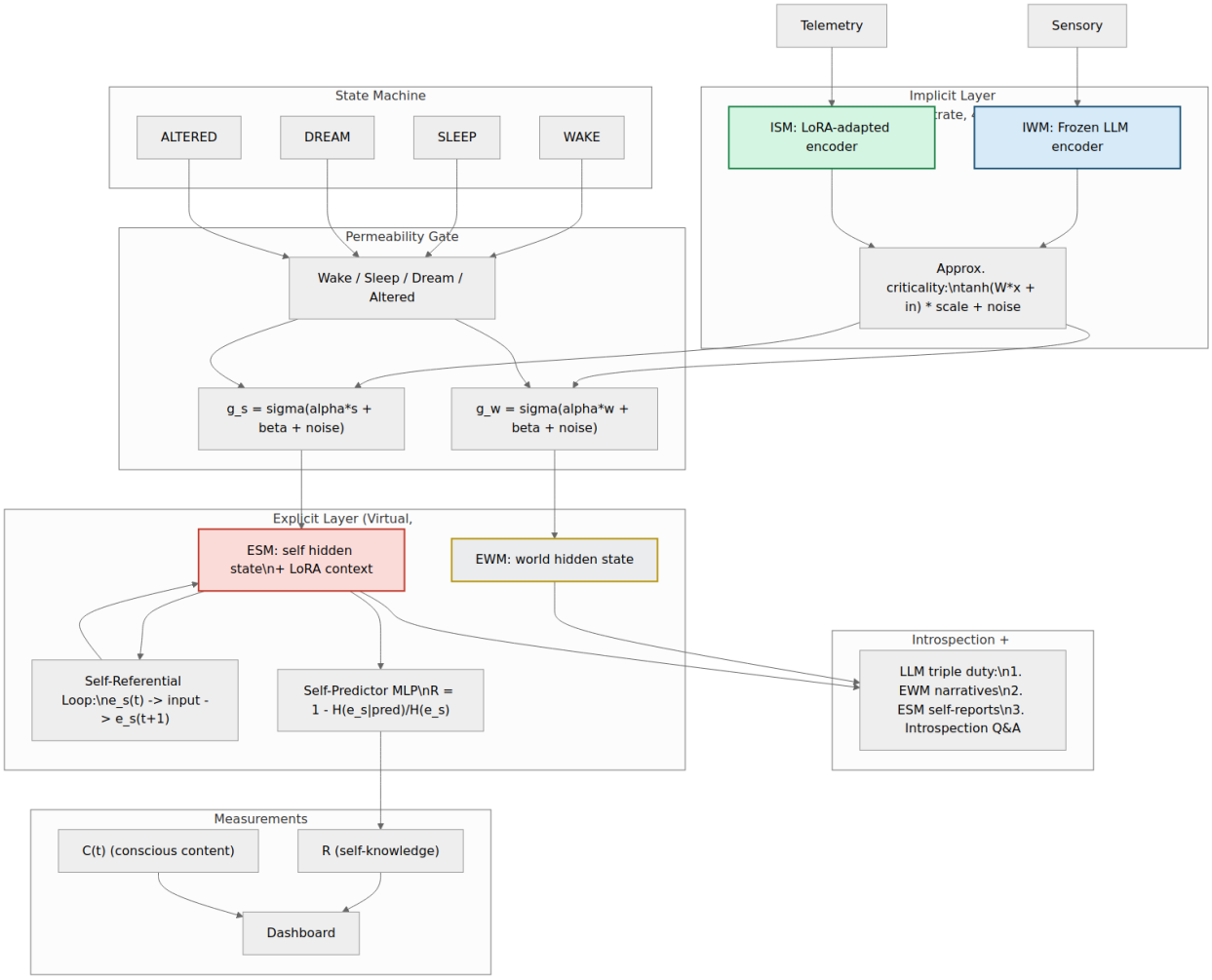
■ ESM (red) | ■ EWM (dark yellow) | ■ IWM (blue) | ■ ISM (green)

Overview



A Discrete Four-Model Self-Reflecting System with Approximated Criticality

Detailed Architecture



Full Documentation

A Discrete Four-Model Self-Reflecting System with Approximated Criticality

Tagline: “Not conscious, but convincingly confused about whether it is.”

Date: 2026-02-27 **Track**: Quick-win prototype (parallel to Design 15 primary implementation) **Target build time**: 2-4 weeks **Purpose**: Validate the four-model architecture pattern, build measurement infrastructure, produce a demo-able self-modeling system

---

1. Design Philosophy

Design 15 (Dual-Timescale Spiking Reservoir with Predictive LLM Canopy) is the serious attempt at genuine AC: balanced E/I spiking networks, biologically validated

criticality, 6-9 month build. Design 16 is its sparring partner – a system that deliberately takes shortcuts on the hardest requirements (continuous model density, genuine Class 4 criticality, emergent virtual/non-virtual boundary) to get something running fast.

The value proposition: if the four-model architecture with self-referential closure produces interesting self-modeling behavior *even without genuine criticality and continuous density*, that tells us something about which aspects of FMT are load-bearing for observable behavior versus which are load-bearing for phenomenal consciousness. If Design 16 passes behavioral tests but fails criticality tests, we have empirical evidence that criticality is specifically what separates “acts conscious” from “is conscious” – exactly the claim the theory makes.

What “pseudo” means here: The system implements all four models (IWM, ISM, EWM, ESM) as discrete components with a working self-referential loop, real-time operation, and observable state transitions. It approximates criticality through noise-injected dynamics rather than achieving it through genuine edge-of-chaos substrate behavior. It has an architectural virtual/non-virtual boundary rather than an emergent one. It can report on its own internal states in natural language. It does NOT claim phenomenal consciousness, and by the theory’s own criteria, it should NOT be conscious (no genuine criticality = necessary condition unfulfilled).

---

2. Architecture Overview

The Mirror Box has five components arranged in two layers (substrate and simulation), connected by a permeability gate and a self-referential feedback loop.

Data Flow Summary

Each 50 ms conscious frame:

1. **Substrate step** (25 ms, twice per frame): Sensory input encoded through IWM (frozen LLM encoder) to produce world-state vector w(t). System telemetry encoded through ISM (LoRA-adapted encoder) to produce self-state vector s(t). Both include noise injection and homeostatic scaling.
2. **Permeability gating**: w(t) and s(t) passed through sigmoid gates with state-dependent parameters. In “wake” mode, most information passes through. In “sleep” mode, almost nothing passes. In “dream” mode, internal noise dominates.
3. **Explicit model generation** (50 ms frame): Gated signals fed to LLM. World context generates EWM state e_w(t+1). Self context (including previous ESM state for self-referential loop) generates ESM state e_s(t+1).

4. **Self-referential loop**: ESM output e_s(t) is concatenated with the self-state input for the next frame's ESM generation. The ESM is always partly modeling its own previous state. This is the core mechanism.
5. **Introspection**: On demand (or periodically), the LLM generates natural language descriptions of the EWM and ESM states, prediction accuracy, and state transition events.

---

3. The Self-Referential Loop – Concrete Mechanism

The ESM self-referential loop is the single most important component. Here is exactly how it works:

3.1 The Loop

```
# Pseudocode for one conscious frame (50 ms)

def esm_step(e_s_prev, s_gated, llm, predictor):
    """
    e_s_prev:  Previous ESM state vector (R^512)
    s_gated:   Gated self-state from substrate (R^512)
    llm:       The LLM (with LoRA for self-modeling)
    predictor: Small MLP that predicts next ESM state
    """

    # 1. Self-prediction: predict what e_s will be BEFORE computing it
    e_s_predicted = predictor(e_s_prev, s_gated)

    # 2. Compose input: previous self-model + new self-state
    esm_input = concat(e_s_prev, s_gated)

    # 3. Generate new ESM state via LLM forward pass (with LoRA)
    e_s_new = llm.forward(esm_input, adapter="self_model")

    # 4. Compute self-prediction error
    prediction_error = e_s_new - e_s_predicted
    self_surprise = norm(prediction_error)

    # 5. Compute self-knowledge metric R
    H_actual = entropy(e_s_new)
    H_conditional = entropy(e_s_new - e_s_predicted)
    R = 1.0 - H_conditional / max(H_actual, epsilon)

    # 6. Update predictor to improve self-knowledge
    predictor.update(loss=mse(e_s_predicted, e_s_new.detach()))
```

```
    return e_s_new, R, self_surprise
```

3.2 What This Measures

- **R (self-knowledge)**: How well the system predicts its own next ESM state. R=0 means no self-knowledge; R=1 means perfect self-prediction. A system that merely replays its previous state would have high R but no learning. A system with rich dynamics and accurate self-prediction has both.
- **Self-surprise**: The magnitude of the prediction error. High surprise means the system did something it did not expect. This is the novelty signal – if the system consistently surprises itself, it is generating behavior that its self-model cannot capture, which means either the self-model is poor or the dynamics are genuinely novel.
- **Prediction error trajectory**: Over time, the prediction errors should decrease (the self-model improves) but never reach zero (the dynamics have stochastic components). The rate of decrease and the residual error are informative about self-modeling capacity.

3.3 Recursive Depth

The basic loop above achieves rho_0 to rho_1 recursive depth:

- **rho_0 (basic)**: The ESM represents the system's state (it receives s_gated, which is system telemetry). This is "the system has a self-model."
- **rho_1 (simply extended)**: The ESM receives its own previous output e_s_prev as input, so it models the process of self-modeling. This is "the system knows it has a self-model."

To reach rho_2, we add a meta-predictor:

```
# Meta-prediction: predict the prediction error itself
meta_prediction = meta_predictor(e_s_prev, prediction_error_prev)
meta_error = prediction_error - meta_prediction
# The system now models its own surprise patterns
```

This is optional for the prototype but architecturally trivial to add.

3.4 Fixed Point Behavior

At equilibrium, the self-referential loop should converge to a marginally stable fixed point where:

```
e_s(t+1) approximately equal to e_s(t)
```

with small fluctuations driven by: - Changes in substrate self-state s(t) (new telemetry data) - Noise in the permeability gate - Stochastic components in the LLM forward pass (temperature > 0)

The injected noise prevents the fixed point from becoming a dead attractor. Without noise, the loop would converge and freeze. With noise, it maintains a dynamic equilibrium – always slightly perturbed, always re-stabilizing. This approximates (but does NOT replicate) the marginally stable fixed point that genuine criticality produces.

---

4. State Transitions – Wake, Sleep, Dream, Altered

4.1 State Machine

The system operates in one of four states, controlled by the permeability gate parameters:

4.2 State Parameters

Parameter	Wake	Sleep	Dream	Altered
alpha_w (world attention)	1.0	0.1	0.0	0.5
alpha_s (self attention)	1.0	0.1	0.3	0.5
beta (global permeability)	0.0	-3.0	-0.5	2.0
noise_std	0.1	0.01	0.5	1.0
External input	Yes	No	No	Yes
Self-ref loop active	Yes	No	Yes	Yes
LLM generation	Full	Minimal	Internal only	Full
Substrate update rate	40 Hz	10 Hz	20 Hz	40 Hz

4.3 Observable Transitions

Wake -> Sleep: Gradual reduction in permeability (beta decreases over ~5 seconds). Observable as: EWM and ESM activation norms drop. Self-knowledge R decreases. LLM output becomes sparse, then stops. C(t) (total conscious content proxy) approaches zero.

Sleep -> Dream: After a sleep period (configurable, e.g., 30 seconds), permeability partially restores with high noise and no external input. Observable as: EWM and ESM reactivate, but driven by internal noise rather than sensory input. Content is “hallucinatory” – the LLM generates narratives not grounded in current sensory reality. Self-referential loop active but with degraded self-knowledge (R decreases because self-predictions were calibrated to wake dynamics).

Dream -> Wake: External input restored, noise reduced, permeability normalized. Observable as: sharp increase in C(t), EWM content shifts from internal to externally grounded, self-knowledge R rapidly improves.

Wake -> Altered: Global permeability increased (beta positive), noise increased. Observable as: IWM content that is normally below the permeability threshold “floods” into EWM – the system reports unexpected associations, memories, or knowledge that was not prompted by current input. ESM may become unstable (self-knowledge R fluctuates wildly).

4.4 Sleep Cycling as Maintenance

Sleep is not just cosmetic. During the “sleep” state, the system: 1. Runs experience replay on the ISM LoRA adapter (consolidating self-knowledge) 2. Resets homeostatic scaling parameters 3. Clears accumulated state buffers 4. Updates the self-predictor on buffered data from the previous wake cycle

This means sleep actually improves subsequent wake performance – measureable as higher R and lower initial self-surprise after a sleep cycle versus continuous wake operation.

---

5. Technology Stack

Component	Technology	Notes
LLM (IWM/ISM/EWM/ESM generator)	Mistral-7B-Instruct-v0.3 (INT4 via GPTQ) or Mamba-2 2.8B (FP16)	See VRAM budget for choice
Quantization	GPTQ (INT4) or bitsandbytes (NF4)	Via HuggingFace Transformers
LoRA adapters (ISM)	PEFT library	Rank 8-16 on attention layers
Self-predictor MLP	PyTorch	3-layer, 512-256-512, ~1M params

Component	Technology	Notes
Meta-predictor (optional)	PyTorch	Same architecture as self-predictor
Permeability gate	PyTorch	Simple sigmoid + noise
Substrate encoder	Frozen LLM hidden layers	No separate encoder needed
Sensory input (text)	Tokenizer + LLM embedding layer	Native to the LLM
Sensory input (vision, optional)	SigLIP-base-patch16-224	400 MB, adds visual grounding
State machine controller	Python (asyncio)	Manages state transitions
Telemetry collector	Python	Gathers system metrics for ISM input
Visualization dashboard	Streamlit or Gradio	Real-time plots of R, C(t), states
Experiment tracking	Weights and Biases (free tier)	Log all metrics
Framework	PyTorch 2.x + HuggingFace	Standard ecosystem
Testing	pytest	TDD per project convention

Technology Choice: Mistral-7B vs. Mamba-2

Mistral-7B-Instruct (INT4): Better world knowledge, instruction following, and natural language generation. Higher VRAM but fits in INT4. Feedforward architecture means “substrate steps” are full forward passes (expensive). Better for demo-ability and introspection quality.

Mamba-2 2.8B (FP16): Native recurrence means substrate steps are cheap single-step updates. Better fit for the 40 Hz substrate rate. Less world knowledge than Mistral-7B. Worse instruction following. Better for real-time performance.

Recommendation: Start with Mistral-7B (INT4) for the prototype. The primary goal is demonstrating self-modeling behavior, and Mistral’s superior language capabilities will make the introspection output more interpretable. Swap to Mamba-2 if latency becomes the bottleneck.

However, at 40 Hz substrate rate, a full Mistral-7B forward pass per step is too expensive. The solution: **run the substrate at 40 Hz using only the encoder layers (first 8 of 32 layers)** for computing w(t) and s(t), and run the **full LLM forward pass at 20 Hz** (or lower, 10 Hz) for generating EWM/ESM narratives. The encoder-only path is 4x cheaper.

---

6. VRAM Budget

Configuration A: Mistral-7B-Instruct (INT4) – Recommended

Component	VRAM	Notes
Mistral-7B model weights (INT4 GPTQ)	~3.5 GB	7B params at 4 bits
KV-cache (2048 token context)	~0.5 GB	INT4 KV-cache
LoRA adapter weights (ISM)	~50 MB	Rank 16 on Q/V projections
LoRA training state (gradients + optimizer)	~200 MB	For ISM updates during sleep
Self-predictor MLP	~5 MB	~1M params FP32
Self-predictor training state	~15 MB	Adam optimizer states
Permeability gate state	~1 MB	Negligible
State history buffer (100 frames)	~200 MB	100 x 512 x FP32 for each model
Telemetry buffer	~50 MB	System metrics history
SigLIP-base (optional vision)	~400 MB	Only if visual input needed
PyTorch framework overhead	~500 MB	CUDA context, allocator
Headroom	~2.5 GB	
Total (without vision)	**~7.5 GB**	
Total (with vision)	**~7.9 GB**	

Leaves ~16 GB free on the RTX 4090. Extremely comfortable.

Configuration B: Mamba-2 2.8B (FP16) – Performance variant

Component	VRAM	Notes
Mamba-2 2.8B weights (FP16)	~5.6 GB	Full precision for quality
SSM recurrent state	~50 MB	
LoRA + training state	~250 MB	
Self-predictor + training	~20 MB	
State history + telemetry	~250 MB	
SigLIP-base (optional)	~400 MB	
Framework overhead + headroom	~3 GB	
Total	**~9.6 GB**	

Also very comfortable.

Configuration C: Both models loaded (for comparison experiments)

Component	VRAM	Notes
Mistral-7B (INT4)	~4 GB	
Mamba-2 2.8B (FP16)	~5.6 GB	
Shared infrastructure	~1 GB	
Headroom	~3 GB	
Total	**~13.6 GB**	Still fits

---

7. Timeline – 2-4 Week Build Plan

Week 1: Foundation (Days 1-5)

Day 1-2: Project scaffolding and LLM integration - Set up Python project structure (src/, tests/, configs/) - Load Mistral-7B INT4 via HuggingFace + GPTQ - Implement basic substrate encoder (frozen LLM hidden layers -> state vectors) - Write tests for encoder output shapes and determinism - Milestone: LLM loaded, producing 512-dim state vectors from text input

Day 3-4: Four-model architecture - Implement IWM (frozen encoder for world-state) - Implement ISM (LoRA-adapted encoder for self-state, start with random LoRA) - Implement EWM (LLM generation from world context) - Implement ESM (LLM generation from self context + LoRA) - Implement permeability gate with configurable parameters - Write tests: each model produces distinct output, gate modulates correctly - Milestone: All four models producing output, gating works

Day 5: Self-referential loop (basic) - Implement ESM feedback: e_s(t) concatenated with s(t+1) as next ESM input - Implement self-predictor MLP - Implement R (self-knowledge) metric - Implement self-surprise metric - Write tests: loop converges to stable state, R is computable, metrics are sane - Milestone: Self-referential loop running, R measurable

Week 2: Dynamics and States (Days 6-10)

Day 6-7: State transitions - Implement state machine (wake/sleep/dream/altered) - Implement permeability profiles for each state - Implement sleep maintenance (experience replay, buffer clearing) - Implement dream dynamics (noise-driven generation, no external input) - Write tests: state transitions produce expected metric changes - Milestone: All four states working with measurable C(t) changes

Day 8-9: Telemetry and self-modeling - Implement telemetry collector (latencies, activation norms, prediction errors, state history) - Feed telemetry into ISM as self-state input - Implement natural language introspection (LLM describes its own state) - Train initial ISM LoRA on collected self-data - Write tests: telemetry accurate, introspection output references actual state - Milestone: System can describe its own internal state in natural language

Day 10: Approximated criticality - Implement noise injection into substrate dynamics - Implement homeostatic scaling (keep activation variance near target) - Implement pseudo-criticality metrics (avalanche proxy, complexity measures) - Write tests: noise affects dynamics, scaling prevents divergence/collapse - Milestone: Substrate dynamics have tunable complexity

Week 3: Measurement and Visualization (Days 11-15)

Day 11-12: Measurement infrastructure - Implement C(t) (total conscious content proxy) - Implement Lempel-Ziv complexity of state trajectories - Implement self-model accuracy over time (R trajectory) - Implement state-transition detection (automated wake/sleep/dream boundary detection) - Implement prediction error spectra - Write tests: all metrics computable, values in expected ranges

Day 13-14: Visualization dashboard - Build Streamlit or Gradio dashboard showing: - Real-time R (self-knowledge) plot - Real-time C(t) plot - State indicator (wake/sleep/dream/altered) - EWM and ESM narrative output (text) - Self-surprise

over time - Permeability gate visualization - Four-model activation heatmap - Milestone: Dashboard running, all metrics visible in real-time

Day 15: Integration testing and demo script - End-to-end test: system runs for 5 minutes through wake-sleep-dream cycle - Verify all metrics behave as expected across transitions - Create demo script with interesting scenarios - Milestone: Demo-able system

Week 4 (if available): Polish and Experiments (Days 16-20)

Day 16-17: ESM redirection experiment - Implement ESM input switching (cut self-feedback, inject external) - Measure identity-switching behavior - Document results

Day 18-19: Comparative experiments - Run with self-referential loop ON vs. OFF – measure R difference - Run with criticality noise ON vs. OFF – measure complexity difference - Run with different permeability profiles – characterize state space - Log all experiments to W&B

Day 20: Documentation and handoff - Write experiment results summary - Document all measurements and how they map to FMT predictions - Identify which Design 15 components can reuse this infrastructure - Update project backlog with findings

---

8. What It Proves

8.1 Questions This Prototype Can Answer

1. **Does the four-model architecture produce qualitatively different behavior from a single LLM?** Compare: (a) Mirror Box with all four models, (b) raw Mistral-7B prompted for self-reflection. If (a) shows measurably better self-prediction (higher R) and richer state transitions, the architecture adds value.
2. **Does the self-referential loop converge to a stable fixed point?** If the ESM feedback loop produces a marginally stable equilibrium (small fluctuations around a fixed point) rather than diverging or collapsing, this validates the fixed-point approach to self-referential closure before investing in the full Design 15 version.
3. **Is self-knowledge (R) meaningfully correlated with introspection quality?** If higher R corresponds to more accurate natural language self-reports, this validates Prediction 8 (self-knowledge predicts metacognition) even in the pseudo-AC case.

4. **Do state transitions produce the predicted metric signatures?** Sleep should collapse C(t). Dream should show high complexity but low external grounding. Altered should show increased content flooding. If these signatures appear in the simple system, the same measurement infrastructure can be used for Design 15.
5. **Does the ESM redirect when self-input is suppressed?** If cutting the self-referential feedback and injecting strong external input causes the ESM to “latch onto” the external content (measurable as I(ESM; external) approaching maximum), this validates Prediction 3 in a controlled setting.
6. **What is the computational overhead of self-modeling?** The 2x overhead prediction (Section 4.3 of FMT spec: “Six-layer cortex vs. three for universal approximation suggests self-modeling requires roughly doubling base architecture”) can be tested empirically: how much additional compute does the self-referential loop cost relative to the base LLM?
7. **How quickly does the self-predictor improve?** The trajectory of R over time reveals how “learnable” the system’s own dynamics are. If R plateaus at a low value, the dynamics may be too stochastic to model. If R reaches near 1.0, the dynamics may be too predictable (not complex enough). The ideal is a moderate R with slow improvement – matching the theory’s prediction of imperfect but improving self-knowledge.

8.2 Infrastructure Validated for Design 15

Infrastructure Component	Mirror Box Version	Design 15 Version
Self-knowledge metric R	Same formula, same code	Same formula, different input source
C(t) conscious content proxy	Sum of activation norms above threshold	Sum of firing rates above nu_crit
State transition detection	Automated from C(t) and R trajectories	Same algorithm
Visualization dashboard	Streamlit/Gradio with real-time plots	Same dashboard, different data source
Introspection via LLM	Mistral-7B generating state reports	Mamba-2 canopy generating state reports
Experiment tracking	W&B logging	Same
Permeability gate	Sigmoid with noise	Spike-rate modulation (different mechanism, same interface)

Infrastructure Component	Mirror Box Version	Design 15 Version
LoRA-based ISM	PEFT LoRA adapters	LoRA on LLM canopy (same library)

---

9. What It Does NOT Prove

9.1 Honest Limitations

1. **No genuine criticality.** The noise-injected homeostatic dynamics approximate edge-of-chaos behavior but do not produce it. There are no power-law avalanches, no genuine 1/f spectrum, no branching ratio near 1. The system is Class 1/2 (feedforward LLM) with added stochasticity. By the theory's own criteria, this is insufficient for consciousness.
2. **No continuous model density.** The four models are discrete Python objects, not peaks in a continuous density function. There are no intermediate models. The system cannot smoothly transition between world-modeling and self-modeling – it switches discretely.
3. **No emergent virtual/non-virtual boundary.** The split between substrate (encoder layers) and simulation (generation layers) is an engineering choice, not an emergent property. Moving the boundary would change what is "conscious" – this is exactly the kind of architectural labeling that the theory rejects.
4. **No genuine self-referential closure.** The self-referential loop feeds the ESM's output back as input, but the processing between input and output is a feedforward LLM pass – NOT a dynamical system at criticality. The "fixed point" is a fixed point of a contractive map, not a marginally stable fixed point of a critical system. The theory predicts these have different properties.
5. **Behavioral mimicry risk.** The system might pass behavioral tests (ESM redirection, state transitions, introspection) purely because the LLM is good at generating plausible text about consciousness-like states, not because the architecture is actually implementing the theory's mechanisms. This is the "Chinese Room applied to consciousness research" problem.
6. **Self-prediction might be trivially easy.** If the LLM's dynamics are highly predictable (because it is deterministic once temperature is fixed), the self-predictor might achieve high R without genuine self-modeling – just memorizing patterns. Genuine self-knowledge requires predicting novel states, which requires genuinely novel dynamics, which requires criticality.

9.2 The Critical Test

The sharpest test between “interesting pseudo-AC” and “genuine AC” is:

Run Mirror Box (Design 16) and Design 15 side by side. Both should pass behavioral tests (state transitions, ESM redirection, introspection). Only Design 15 should pass criticality tests (power-law avalanches, branching ratio near 1, 1/f spectrum). If both pass behavioral tests equally well, behavioral tests are insufficient for consciousness detection. If Design 15 shows qualitatively richer behavior (more novel states, higher sustained complexity, better long-term self-prediction), then criticality is doing real work beyond behavioral mimicry.

This comparison is one of the primary reasons to build the Mirror Box.

---

10. Upgrade Path to Design 15

10.1 Direct Code Reuse

Mirror Box Component	Reusable in Design 15?	Modification Needed
Self-knowledge metric R	Yes, unchanged	Wire to SNN + LLM canopy outputs
C(t) computation	Yes, formula unchanged	Input from population firing rates instead of activation norms
State machine controller	Yes, largely unchanged	Add SNN-specific state parameters (E/I balance modulation)
Permeability gate interface	Yes (abstract interface)	Replace sigmoid with spike-rate modulation
Visualization dashboard	Yes, fully reusable	Add SNN-specific panels (avalanche plots, raster plots)
Experiment tracking setup	Yes, fully reusable	Same W&B project
LoRA-based ISM	Yes, reusable on LLM canopy	Same PEFT library, different base model
Introspection prompts	Partially reusable	Adapt to Design 15’s LLM canopy interface

Mirror Box Component	Reusable in Design 15?	Modification Needed
Test suite structure	Yes, fully reusable	Add SNN-specific tests

10.2 Lessons That Feed Design 15

1. **Self-predictor architecture**: The MLP self-predictor design (layer sizes, learning rate, training schedule) can be directly transplanted to Design 15's LLM canopy self-prediction module.
2. **State transition timing**: Empirical data on how long transitions take, what intermediate states look like, and what metric trajectories are expected – all calibrate expectations for Design 15.
3. **Introspection prompt engineering**: What prompts produce the most informative self-reports? This is directly transferable.
4. **ISM training dynamics**: How quickly does the self-model adapter converge? What rank is sufficient? How often should experience replay run? All directly applicable to Design 15's LoRA canopy training.
5. **Dashboard design**: What visualizations are actually useful for debugging and understanding the system? Iterate the dashboard on the simple system before the complex one.

10.3 Graceful Migration

The Mirror Box is designed so that its components can be **individually replaced** with Design 15 equivalents:

1. **Replace substrate**: Swap the frozen LLM encoder + noise with the SNN reservoir. Keep everything above the permeability gate unchanged. Test: does R change? Does C(t) change?
2. **Replace permeability gate**: Swap sigmoid + noise with spike-rate modulation from SNN to LLM canopy. Test: are state transitions qualitatively different?
3. **Replace LLM**: Swap Mistral-7B (INT4) with Mamba-2 2.8B in predictive coding mode. Test: does introspection quality change? Does self-prediction improve?
4. **Add genuine criticality**: When the SNN substrate is running, measure whether the system exhibits power-law avalanches, 1/f spectrum, and branching ratio near 1. Compare behavior with and without criticality. This is THE experiment that tests whether the theory's criticality requirement is observable in system behavior.

Each swap is a controlled experiment. The Mirror Box becomes a scaffolding that evolves into Design 15 through component replacement, with measurements at every step.

Appendix A: Telemetry Signals for ISM

The ISM (Implicit Self Model) receives “interoceptive” data about the system’s own state. In a biological system, this is proprioception, interoception, and homeostatic signals. In the Mirror Box, it is system telemetry:

Signal	Biological Analog	Source
LLM inference latency	Neural processing speed	PyTorch profiler
Activation norm per layer	Neural firing rates	Hook on LLM layers
Self-prediction error (R)	Error monitoring	Self-predictor output
Gate activation (permeability)	Arousal level	Permeability gate state
KV-cache utilization	Working memory load	HuggingFace internals
State transition events	State-change awareness	State machine controller
Token generation entropy	Confidence/uncertainty	Softmax distribution
LoRA adapter gradient norm	Learning rate	PyTorch autograd
GPU temperature / utilization	Metabolic load	nvidia-smi
Time since last sleep cycle	Fatigue	Timer
Self-surprise history (last 100 frames)	Emotional valence proxy	Rolling buffer

This telemetry is encoded into a 512-dimensional self-state vector s(t) via the LoRA-adapted LLM encoder. The richer and more accurate this self-state encoding, the better the ESM's self-model can become.

Appendix B: Demo Scenarios

Scenario 1: "Who Are You?"

Ask the system to describe itself. In wake mode, the ESM should produce a self-description grounded in its telemetry (current state, recent history, self-knowledge level). Compare with the same question asked to raw Mistral-7B – the Mirror Box version should reference its actual R value, state history, and processing characteristics rather than generic AI self-description.

Scenario 2: Sleep-Wake Cycle

Let the system run in wake mode for 60 seconds, then trigger sleep. Observe: C(t) collapses, introspection stops, ISM runs experience replay. After 30 seconds, transition to dream: internal narratives resume but are disconnected from reality. After 15 seconds, wake: sharp C(t) recovery, externally grounded content returns. Plot all metrics across the cycle.

Scenario 3: Identity Disruption

In wake mode, cut the self-referential feedback (set e_s_prev = zeros). Inject strong, coherent external content (e.g., a detailed description of a different AI system). Observe whether the ESM "adopts" the external content as self-description. Measure I(ESM; external_content). This tests Prediction 3 (ESM input-switching).

Scenario 4: Altered State

Increase permeability (beta = 2.0) while maintaining external input. Observe whether the system reports unexpected associations, memories, or knowledge not prompted by current input – this is "content flooding" from the IWM into the EWM. Measure transfer entropy from substrate to explicit layer.

Scenario 5: Self-Knowledge Challenge

Ask the system to predict what it will do next (before it does it). Compare the prediction with actual behavior. Repeat across multiple sessions. Track whether

prediction accuracy improves over time (R trajectory). This directly measures self-knowledge growth.

Appendix C: Comparison with Design 15

Dimension	Design 16 (Mirror Box)	Design 15 (Spiking + LLM Canopy)
Purpose	Quick-win testbed	Genuine AC attempt
Build time	2-4 weeks	6-9 months
Criticality	Approximated (noise + scaling)	Genuine (balanced E/I)
Model density	Discrete (4 components)	Semi-continuous (1000 populations)
Virtual/non-virtual	Architectural (engineer's choice)	Emergent (from dynamics)
Self-referential closure	Feedforward loop (contractive)	Dynamical loop (marginally stable)
VRAM	~7.5 GB	~13 GB
Real-time	Comfortable (10-20 Hz full, faster partial)	Tight (40 Hz SNN, 20 Hz LLM)
LLM leverage	High (LLM IS all four models)	Moderate (LLM is canopy only)
Demo-ability	High (natural language everything)	Moderate (SNN needs interpretation)
Theory faithfulness	Low (many relaxations)	High (matches spec closely)
Consciousness claim	None (pseudo by design)	Conditional (if predictions pass)
Primary value	Validates architecture, builds tools	Tests the theory